

AIML SESSION 08 ASSIGNMENT SUBMISSION

Name of Student : Vaibhav Vijay Kate

Domain/Subject : AIML (Artificial Intelligence & Machine Learning)

College : Government Polytechnic College Washim

Branch : Information Technology

Session : 08

Q1

Q1. Creating Sets)

Write a program to create the following sets:

- a) A set with 5 integers.
- b) A set with mixed data types (integer, string, float).
- c) An empty set using the correct method.
- d) A set from the string "hello" using the set() constructor.

Print each set and its type. Add comments explaining why sets automatically remove duplicates.

Program Code:

```
# a) A set with 5 integers.
```

```
set1 = {1, 2, 3, 4, 5}
```

```
print(type(set1), set1)
```

```
# b) A set with mixed data types (integer, string, float).
```

```
set2 = {1, "Vaibhav", 1.2}
```

```
print(type(set2), set2)
```

```
# c) An empty set using the correct method.
```

```
empty_set = set()
```

```
print(type(empty_set), empty_set)
```

```
# d) A set from the string "hello" using the set() constructor.
```

```
set4 = set("hello")
```

```
print(type(set4), set4)
```

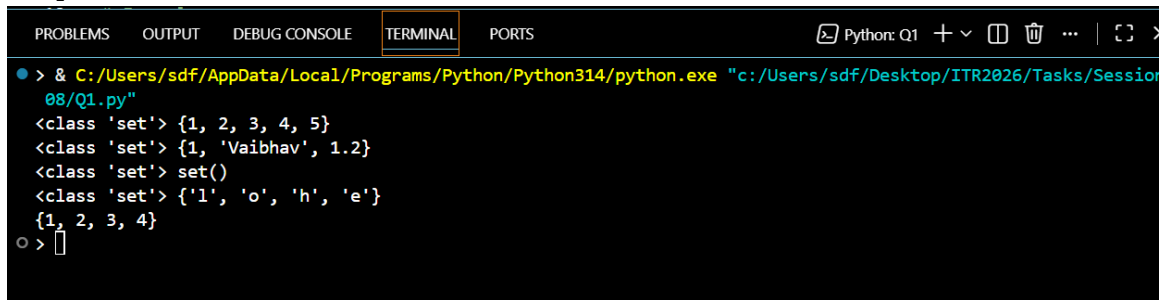
```
# Sets automatically remove duplicates because a set stores only unique values.
```

```
# Example:
```

```
duplicate_set = {1, 2, 2, 3, 3, 4}
```

```
print(duplicate_set)
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python: Q1 + - [ ] [ ] ... | [ ] >

> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session08/Q1.py"
<class 'set'> {1, 2, 3, 4, 5}
<class 'set'> {1, 'Vaibhav', 1.2}
<class 'set'> set()
<class 'set'> {'l', 'o', 'h', 'e'}
{1, 2, 3, 4}
> [ ]
```

Q2

Q2. Characteristics of Sets)

Create a set: numbers = 10, 20, 30, 20, 40, 10, 50

Print the set and observe the output.

Demonstrate that sets are unordered by printing the set multiple times.

Try to access the first element using indexing (numbers[0] and explain the error in comments.

Program Code:

```
# Create a set: numbers = {10, 20, 30, 20, 40, 10, 50}
numbers = {10, 20, 30, 20, 40, 10, 50}
```

```
# Print the set and observe the output.
print(numbers)
```

```
# Demonstrate that sets are unordered by printing the set multiple times.
print(numbers)
print(numbers)
print(numbers)
print(numbers)
print(numbers)
```

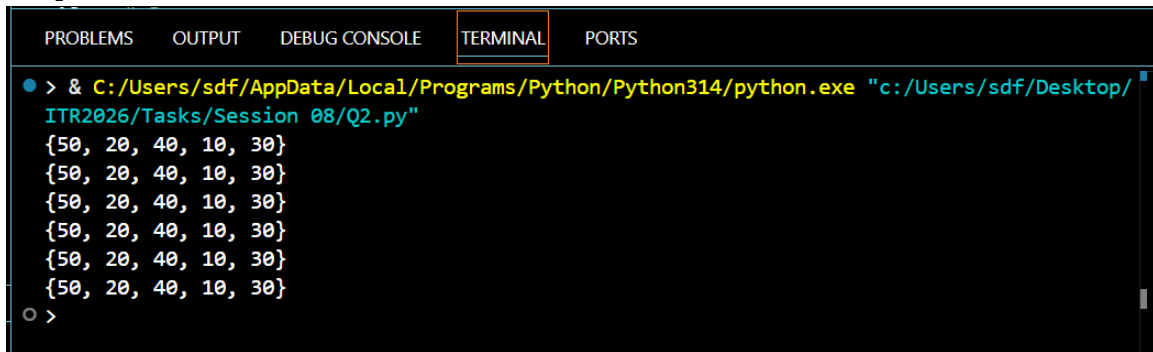
```
# Try to access the first element using indexing (numbers[0]) and explain the
# print(numbers[0])
```

```
# Reason:
```

```
# set are unordered that doesn't follow indexing
```

```
# That is numbers[0], numbers[1] indexing operations not allowed .
```

Output Screenshot:



The screenshot shows a terminal window with a dark background and light-colored text. At the top, there is a tab bar with five tabs: "PROBLEMS", "OUTPUT", "DEBUG CONSOLE", "TERMINAL" (which is selected and highlighted with a blue border), and "PORTS". Below the tab bar, the terminal displays the command to run a Python script: `> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q2.py"`. The output of the script consists of six lines, each displaying a list: `{50, 20, 40, 10, 30}`. At the bottom of the terminal, there is a prompt character `>` and a small circle icon to its left.

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q2.py"
{50, 20, 40, 10, 30}
{50, 20, 40, 10, 30}
{50, 20, 40, 10, 30}
{50, 20, 40, 10, 30}
{50, 20, 40, 10, 30}
{50, 20, 40, 10, 30}
○ >
```

Q3

Q3. Membership Testing)

Write a program that:

Takes a set of 5 colors as input from the user.

Asks the user to enter a color name.

Checks whether the color is present in the set using in and not in.

Prints appropriate messages.

Program Code:

```
# (Membership Testing)
```

```
# Write a program that:
```

```
# Takes a set of 5 colors as input from the user.
```

```
# Asks the user to enter a color name.
```

```
# Checks whether the color is present in the set using in and not in.
```

```
# Prints appropriate messages.
```

```
# Takes a set of 5 colors as input from the user.
```

```
colors = set()
```

```
for i in range(5):
```

```
    color = input("Enter a color: ")
```

```
    colors.add(color)
```

```
# Asks the user to enter a color name.
```

```
search_color = input("Enter a color to search: ")
```

```
# Checks whether the color is present in the set using in and not in.
```

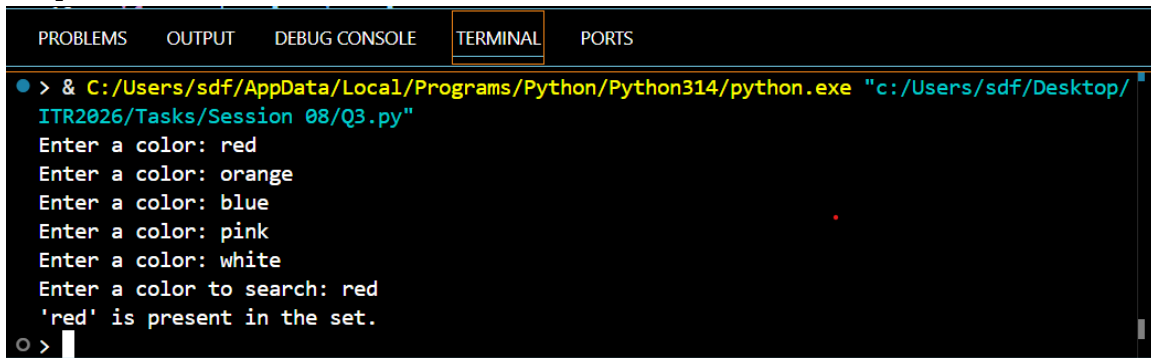
```
if search_color in colors:
```

```
    print(f'{search_color}' is present in the set.)
```

```
else:
```

```
    print(f'{search_color}' is not present in the set.)
```

Output Screenshot:



```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q3.py"
Enter a color: red
Enter a color: orange
Enter a color: blue
Enter a color: pink
Enter a color: white
Enter a color to search: red
'red' is present in the set.
○ >
```

Q4

Q4. (add(), remove(), discard())

Create a set: fruits = {'apple', 'banana', 'mango'}

Perform the following operations and print the set after each step:

a) Add 'orange' using add().

b) Add 'banana' again (observe the result).

Session 08 (AIML) - Assignment Questions 1

c) Remove 'mango' using remove().

d) Try to remove 'grape' using discard() (no error should occur).

Program Code:

```
# Creating a Set
```

```
fruits = {'apple', 'banana', 'mango'}
```

```
# Perform the following operations and print the set after each step:
```

```
# a) Add 'orange' using add().
```

```
fruits.add('orange')
```

```
print(fruits)
```

```
# b) Add 'banana' again (observe the result).
```

```
fruits.add('banana')
```

```
# c) Remove 'mango' using remove().
```

```
fruits.remove('mango')
```

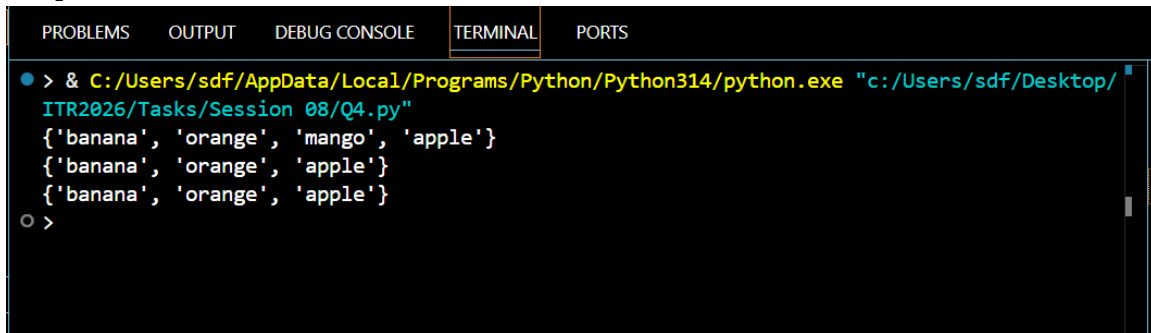
```
print(fruits)
```

```
# d) Try to remove 'grape' using discard() (no error should occur).
```

```
fruits.discard('grape')
```

```
print(fruits)
```


Output Screenshot:



```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q4.py"
{'banana', 'orange', 'mango', 'apple'}
{'banana', 'orange', 'apple'}
{'banana', 'orange', 'apple'}
>
```

Q5

Q5. (pop() and clear())

Create a set: s = 100, 200, 300, 400, 500

Use pop() to remove and print one element.

Print the set after popping.

Clear the entire set using clear().

Print the final empty set.

Add comments explaining the difference between remove(), discard(), and pop().

Program Code:

```
# Creating a set
s = {100, 200, 300, 400, 500}

# Use pop() to remove and print one element.
removed_element = s.pop()
print("Removed element:", removed_element)

# Print the set after popping.
print("Set after pop :",s)

# Clear the entire set using clear().
s.clear()

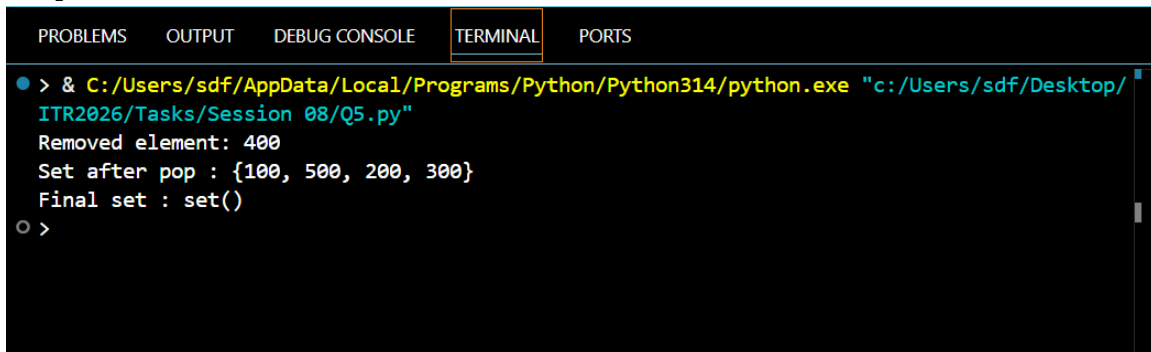
# Print the final empty set.
print("Final set :",s)

# remove(x):
# Removes the specified element.
# Raises KeyError if the element is not found.

# discard(x):
# Removes the specified element.
# Does not raise an error if the element is not found.

# pop():
# removes an arbitrary element (sets have no indexing, so there is no "last" element).
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q5.py"
Removed element: 400
Set after pop : {100, 500, 200, 300}
Final set : set()
○ >
```

Q6

Q6. (update() and copy())

Create two sets:

set1 = 1, 2, 3

set2 = 3, 4, 5, 6

Make a copy of set1.

Update set1 with elements from set2 using update().

Print both the original copy and the updated set.

Program Code:

```
# Creating Sets
```

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5, 6}
```

```
# Copy of set1
```

```
c1 = set1.copy()
```

```
# Update set1 with set2
```

```
set1.update(set2)
```

```
# Print results
```

```
print("Original Copy:", c1)
```

```
print("Updated Set1:", set1)
```

Output Screenshot:



```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q6.py"
Original Copy: {1, 2, 3}
Updated Set1: {1, 2, 3, 4, 5, 6}
>
```

Q7

Q7. Union and Intersection)

Create two sets:

A = 1, 2, 3, 4, 5

B = 4, 5, 6, 7, 8

Print the union of A and B (using both .union() and operator).

Print the intersection of A and B (using both .intersection() and & operator).

Add comments explaining what each operation does.

Program Code:

```
# Creating Sets
```

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
# UNION
```

```
# Union combines all unique elements from both sets.
```

```
union1 = A.union(B)
```

```
union2 = A | B
```

```
print("Union using union():", union1)
```

```
print("Union using | operator:", union2)
```

```
# INTERSECTION
```

```
# Intersection returns only the common elements present in both sets.
```

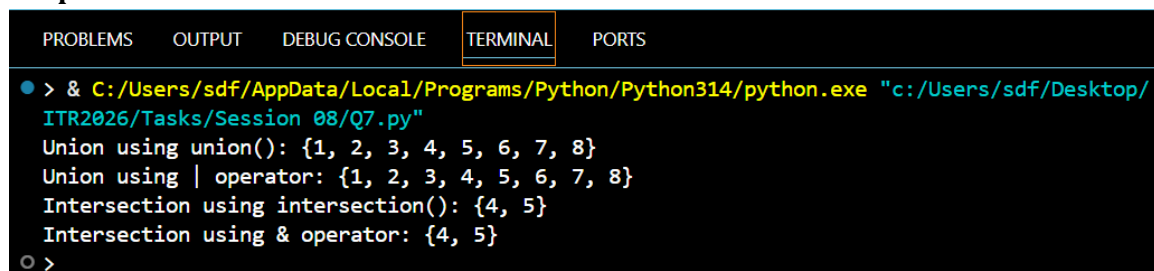
```
intersection1 = A.intersection(B)
```

```
intersection2 = A & B
```

```
print("Intersection using intersection():", intersection1)
```

```
print("Intersection using & operator:", intersection2)
```

Output Screenshot:



```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q7.py"
Union using union(): {1, 2, 3, 4, 5, 6, 7, 8}
Union using | operator: {1, 2, 3, 4, 5, 6, 7, 8}
Intersection using intersection(): {4, 5}
Intersection using & operator: {4, 5}
```

Q8

Q8. Combined Methods)

Write a program that:

Takes 6 numbers as input from the user and stores them in a set (duplicates should be removed automatically).

Session 08 (AIML) - Assignment Questions 2

Adds two more numbers using add().

Removes one number safely using discard().

Prints the final set and its length using len().

Program Code:

```
# Taking 6 numbers as input and storing them in a set
s = set()

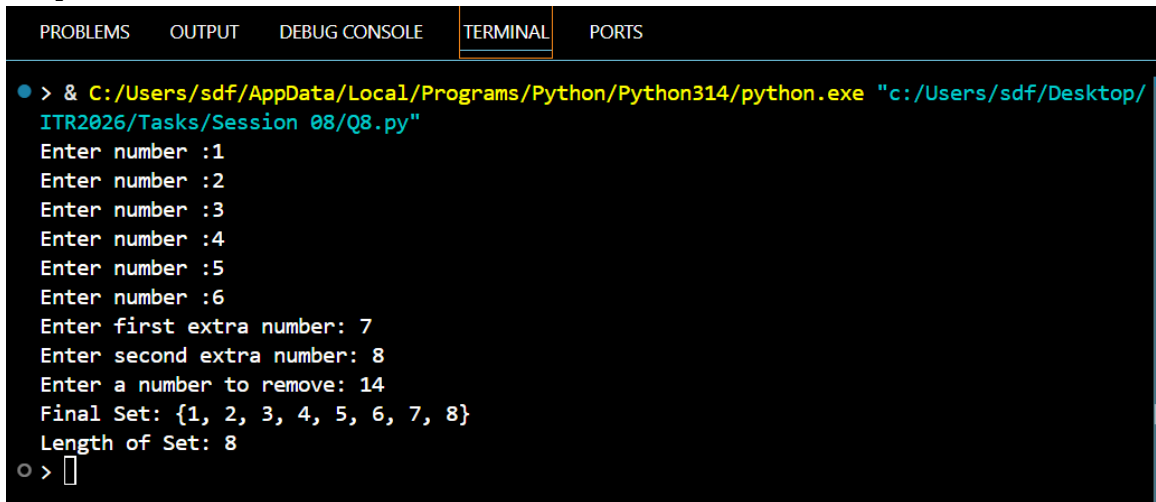
for i in range(6):
    num = int(input("Enter number :"))
    s.add(num) # Duplicates will be removed automatically

# Adding two more numbers
s.add(int(input("Enter first extra number: ")))
s.add(int(input("Enter second extra number: ")))

# Removing one number safely using discard()
remove_num = int(input("Enter a number to remove: "))
s.discard(remove_num)

# Printing final set and its length
print("Final Set:", s)
print("Length of Set:", len(s))
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q8.py"
Enter number :1
Enter number :2
Enter number :3
Enter number :4
Enter number :5
Enter number :6
Enter first extra number: 7
Enter second extra number: 8
Enter a number to remove: 14
Final Set: {1, 2, 3, 4, 5, 6, 7, 8}
Length of Set: 8
○ > 
```

Q9

Q9. Practical Application)

Write a program to remove duplicate marks from a list of student scores using a set:

```
scores = 85, 92, 78, 92, 85, 88, 95, 78
```

Convert the list to a set.

Print the unique scores.

Convert the set back to a sorted list and print it.

Print the total number of unique scores.

Program Code:

```
# Q9. (Practical Application)
```

```
# Original list of student scores
```

```
scores = [85, 92, 78, 92, 85, 88, 95, 78]
```

```
# Convert list to set (removes duplicates)
```

```
unique_scores = set(scores)
```

```
# Print unique scores
```

```
print("Unique Scores:", unique_scores)
```

```
# Convert set back to a sorted list
```

```
sorted_scores = sorted(unique_scores)
```

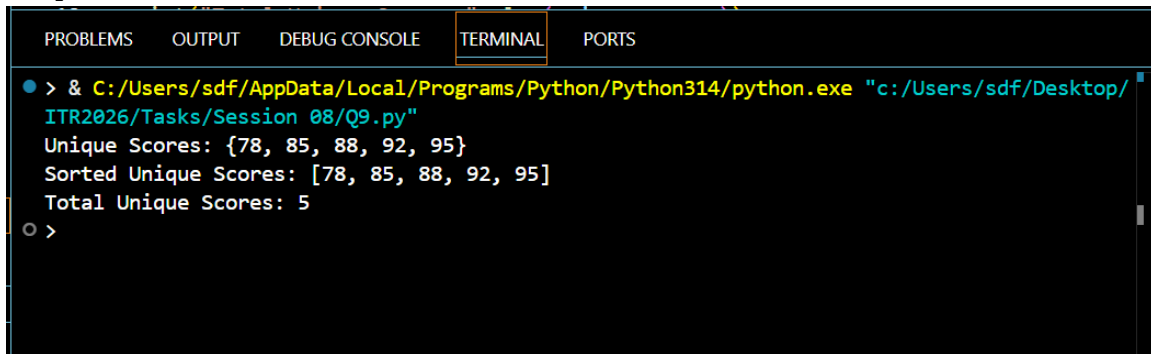
```
# Print sorted list
```

```
print("Sorted Unique Scores:", sorted_scores)
```

```
# Print total number of unique scores
```

```
print("Total Unique Scores:", len(unique_scores))
```


Output Screenshot:



The screenshot shows a terminal window with a dark background and light-colored text. At the top, there is a horizontal bar with five tabs: "PROBLEMS", "OUTPUT", "DEBUG CONSOLE", "TERMINAL" (which is highlighted with a blue border), and "PORTS". Below the tabs, the terminal displays the following output:

```
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q9.py"
Unique Scores: {78, 85, 88, 92, 95}
Sorted Unique Scores: [78, 85, 88, 92, 95]
Total Unique Scores: 5
○ >
```

The output shows the execution of a Python script that calculates unique scores. The results are displayed as a set, a sorted list, and a count.

Q10

Q10. Mini Project - Combined Concepts)

Create a Unique Item Collector program using sets:

Repeatedly ask the user to enter items (e.g., favorite fruits, colors, or numbers).

Store all entered items in a set (duplicates should be ignored).

Provide a menu with options:

Add item

Remove item (use discard)

Show all unique items

Check if an item exists

Clear all items

Exit

Use a while loop for the menu and appropriate set methods. Add meaningful comments.

Session 08 (AIML) - Assignment Questions 3

Program Code:

```
# Unique Item Collector Program using Sets
```

```
# Empty set to store unique items
```

```
items = set()
```

```
while True:
```

```
    # Display Menu
```

```
    print("\nUnique Item Collector ")
```

```
    print("1. Add Item")
```

```
    print("2. Remove Item")
```

```
    print("3. Show All Unique Items")
```

```
    print("4. Check if an Item Exists")
```

```
    print("5. Clear All Items")
```

```
    print("6. Exit")
```

```
    choice = input("Enter your choice (1-6): ")
```

```
    # Option 1: Add Item
```

```
    if choice == "1":
```

```
        item = input("Enter item to add: ")
```

```
        items.add(item)
```

```
        print(f'{item}' added successfully!)
```

```

# Option 2: Remove Item
elif choice == "2":
    item = input("Enter item to remove: ")
    items.discard(item) # Safe removal, no error if item doesn't exist
    print(f"'{item}' removed (if it existed).")

# Option 3: Show All Unique Items
elif choice == "3":
    if len(items) == 0:
        print("No items available.")
    else:
        print("Unique Items:", items)

# Option 4: Check if an Item Exists
elif choice == "4":
    item = input("Enter item to check: ")
    if item in items:
        print(f"'{item}' exists in the set.")
    else:
        print(f"'{item}' does not exist in the set.")

# Option 5: Clear All Items
elif choice == "5":
    items.clear()
    print("All items have been cleared.")

# Option 6: Exit Program
elif choice == "6":
    print("Exiting program. Goodbye!")
    break

# Invalid Choice
else:
    print("Invalid choice! Please enter a number between 1 and 6.")

```

Output Screenshot:

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 08/Q10.py"
```

```
Unique Item Collector
1. Add Item
2. Remove Item
3. Show All Unique Items
4. Check if an Item Exists
5. Clear All Items
6. Exit
Enter your choice (1-6): 3
No items available.
```

```
Unique Item Collector
1. Add Item
2. Remove Item
3. Show All Unique Items
4. Check if an Item Exists
5. Clear All Items
6. Exit
Enter your choice (1-6): 1
Enter item to add: 12
'12' added successfully!
```

```
Unique Item Collector
1. Add Item
2. Remove Item
3. Show All Unique Items
4. Check if an Item Exists
5. Clear All Items
6. Exit
Enter your choice (1-6): █
```